Q6 – Recursion and Its Limits

Recurrence

The number of ways to climb n steps using 1 or 2 steps is W(n)=W(n–1)+W(n–2), with W(0)=1 and W(1)=1. For n=5, the recursive branches repeatedly calculate W(3), W(2), and smaller values.

Naive and memoised

Plain recursion has exponential time O(ϕ^n) because it repeats subproblems. Memoisation stores each W(k), reducing time to O(n) and cache space to O(n). The iterative version also takes O(n) time and O(1) extra space.

Deployment

A recursive memoised function at n=1,000 may hit Python's recursion limit because the call depth is proportional to n. Raising the limit can risk stack exhaustion; an iterative loop is the safer production choice for large n. The code reports results and the current recursion limit.

Python Program

```
from functools import lru_cache
import sys

calls = 0
def ways_naive(n):
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_naive(n - 1) + ways_naive(n - 2)

@lru_cache(maxsize=None)
def ways_memo(n):
    if n <= 1:
        return 1
    return ways_memo(n - 1) + ways_memo(n - 2)

def ways_iterative(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    a, b = 1, 1
    for _ in range(n):
        a, b = b, a + b
    return a

if __name__ == "__main__":
    for n in range(1, 7):
        print(n, ways_memo(n))
    for n in (10, 20, 25, 30):
        calls = 0
        answer = ways_naive(n)
        print("naive", n, answer, "calls", calls)
        print("memo", n, ways_memo(n))
    print("iterative 1000 digits:", len(str(ways_iterative(1000))))
    print("recursion limit:", sys.getrecursionlimit())
```